

Detection of Freezing of Gait from Lower Back Accelerometry

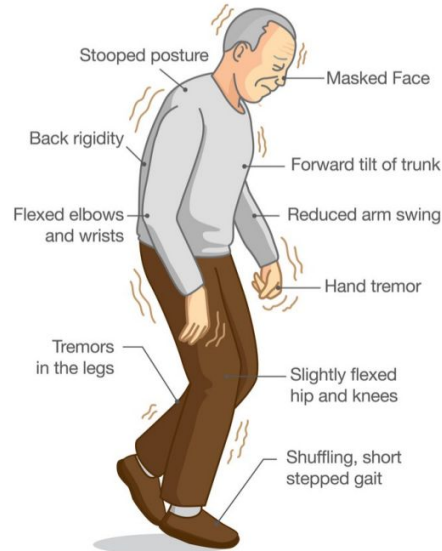
Souvick Chakraborty & Robbin Romijnders

Opencampus - Advanced Time Series Prediction

- Introduction
- Data (pre)processing
- Models
 - Basic models
 - Advanced models
- Conclusion

Introduction – Parkinson's disease

Parkinson's Disease Symptoms



[Source](#)

VIEWPOINT

E. Ray Dorsey, MD

Center for Health and Technology,
Department of Neurology, University of Rochester Medical Center, Rochester, New York.

Bastiaan R. Bloem, MD, PhD

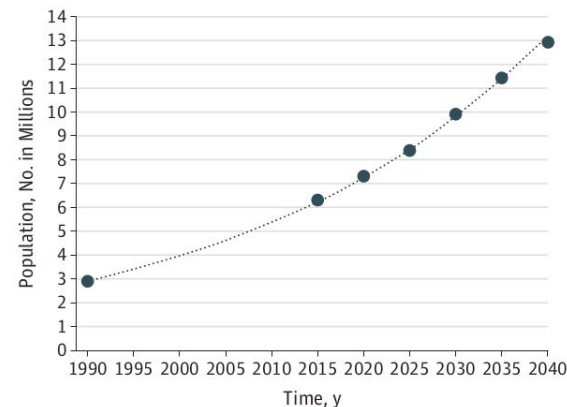
Radboud University Medical Center, Donders Institute for Brain, Cognition and Behavior, Department of Neurology, Nijmegen, the Netherlands.

The Parkinson Pandemic—A Call to Action

Pandemics are usually equated with infectious diseases such as Zika, influenza, and HIV. However, an imminent noninfectious pandemic, Parkinson disease (PD), requires immediate action.

Neurological disorders are now the leading cause of disability in the world.¹ Among these neurological disorders, the fastest growing is PD, whose growth is surpassing that of Alzheimer disease.¹ From 1990 to 2015, the prevalence of, and thus disability and deaths owing to, PD more than doubled.¹ The Global Burden of Disease Study estimates that 6.2 million individuals currently have PD. Because the incidence of PD increases sharply with age and because the world's population is aging, the number of individuals affected is poised for exponential growth (Figure). Conservatively applying worldwide prevalence data from a 2014 meta-analysis² to projections of the world's future population,³ the number of people with PD will double from 6.9 million in 2015

Figure. Estimated and Projected Number of Individuals With Parkinson Disease, 1990-2040



Sources: Global Burden of Disease Study (1990 and 2015) and projections based on published² and public³ sources.

Introduction

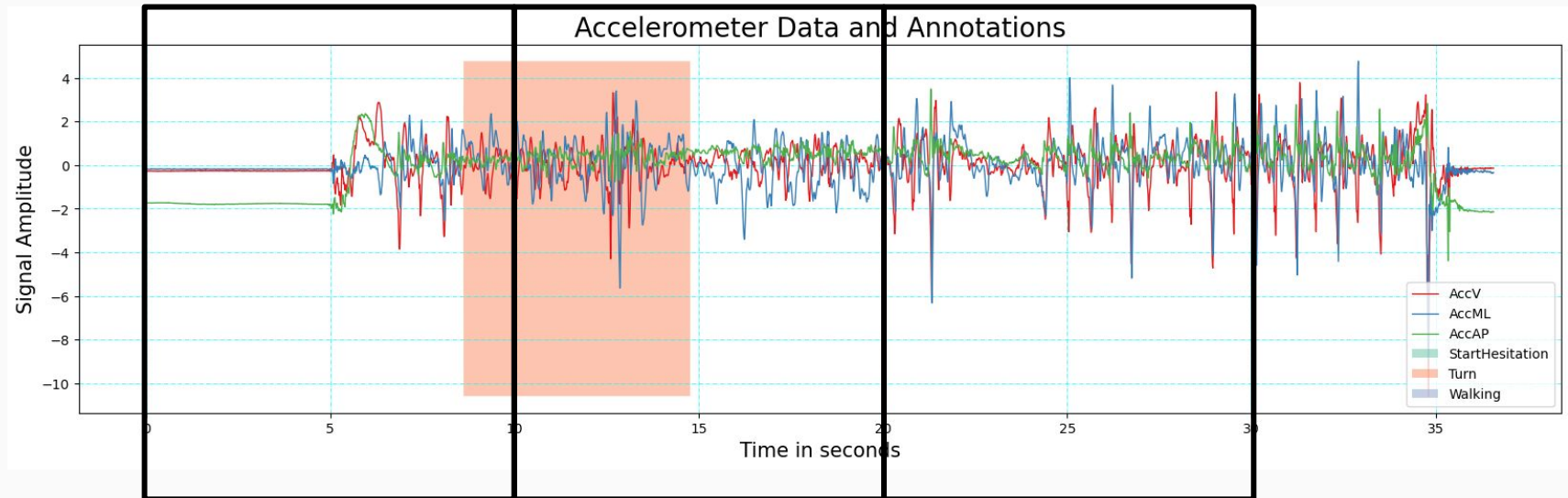
Freezing of gait (FOG) is defined as “a brief, episodic absence or marked reduction of forward progression of the feet despite the intention to walk”¹. FOG can occur in several neurological disorders^{2–6} but is most common in Parkinson disease (PD) and atypical forms of parkinsonism^{7–9}, and is considered to be among the most debilitating motor symptoms¹⁰. The prevalence of FOG is higher among people with more advanced disease; approximately 50% of people with advanced PD experience FOG^{8,11,12}. FOG increases the risk of falls and fall-related injuries, and leads to a decline in functional mobility and independence, with negative effects on quality of life^{13,14}.



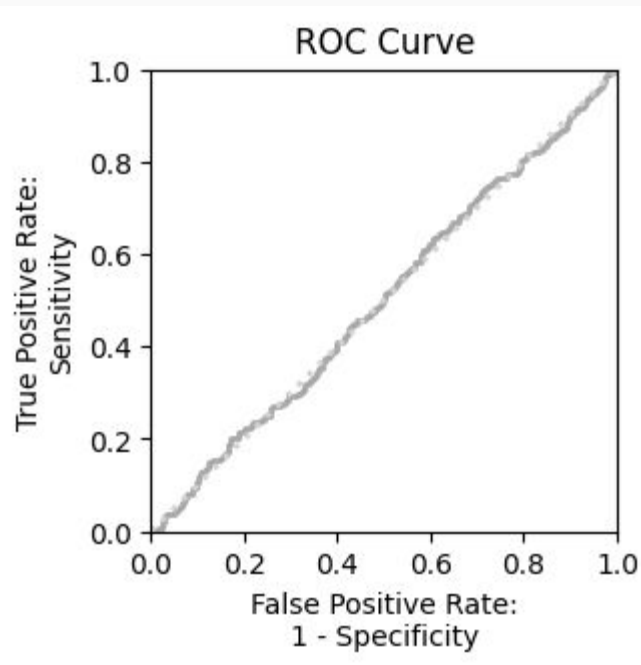
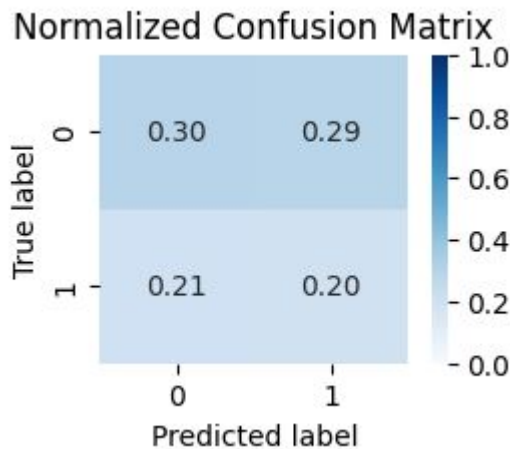
THE MICHAEL J. FOX FOUNDATION · RESEARCH CODE COMPETITION · 2 YEARS AGO

Parkinson's Freezing of Gait Prediction

Event detection from wearable sensor data



Flipping a coin: 50/50



Accuracy
0.50

Time series forest classifier: sktime

		acc_x			acc_y			acc_z		
True label	0	0.30	0	0	0	0	0	0	0	0
	1	0.21	0	0	0	0	0	0	0	0
	2	0.30	0	0	0	0	0	0	0	0
	3	0.30	0	0	0	0	0	0	0	0
	4	0.30	0	0	0	0	0	0	0	0
	...	...	...	...	...	...	...	...	...	...
	4131	0.30	0	0	0	0	0	0	0	0
	4132	0.30	0	0	0	0	0	0	0	0
	4133	0.30	0	0	0	0	0	0	0	0
	4134	0.30	0	0	0	0	0	0	0	0
	4135	0.30	0	0	0	0	0	0	0	0
Pred		0	0	0	0	0	0	0	0	0

cy
8

(Deep) Conv-LSTM Model: PyTorch

```
DeepConvLSTM(  
    (batchnorm1): BatchNorm1d(3, eps=1e-05, momentum=0.1, affine=True,  
    track_running_stats=True)  
    (conv1): Conv1d(3, 64, kernel_size=(5,), stride=(1,), padding=(2,))  
    (batchnorm2): BatchNorm1d(64, eps=1e-05, momentum=0.1, affine=True,  
    track_running_stats=True)  
    (lstm): LSTM(64, 128, num_layers=2, batch_first=True)  
    (classifier): Sequential(  
        (0): Linear(in_features=128, out_features=64, bias=True)  
        (1): ReLU()  
        (2): Linear(in_features=64, out_features=2, bias=True)  
    )  
)
```

Accuracy
0.74

Random Forest

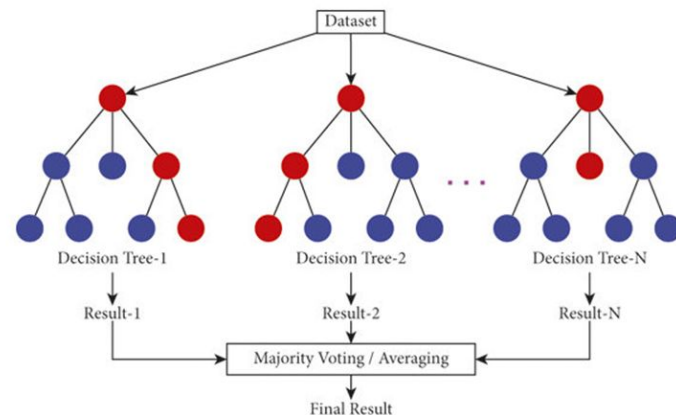
```
X_train_flat = X_train.reshape(X_train.shape[0], -1) # (3308, 7680)
X_test_flat = X_test.reshape(X_test.shape[0], -1) # (960, 7680)
X_train_flat.shape, y_train.shape, X_test_flat.shape, y_test.shape
```

```
((3308, 7680), (3308,), (960, 7680), (960,))
```

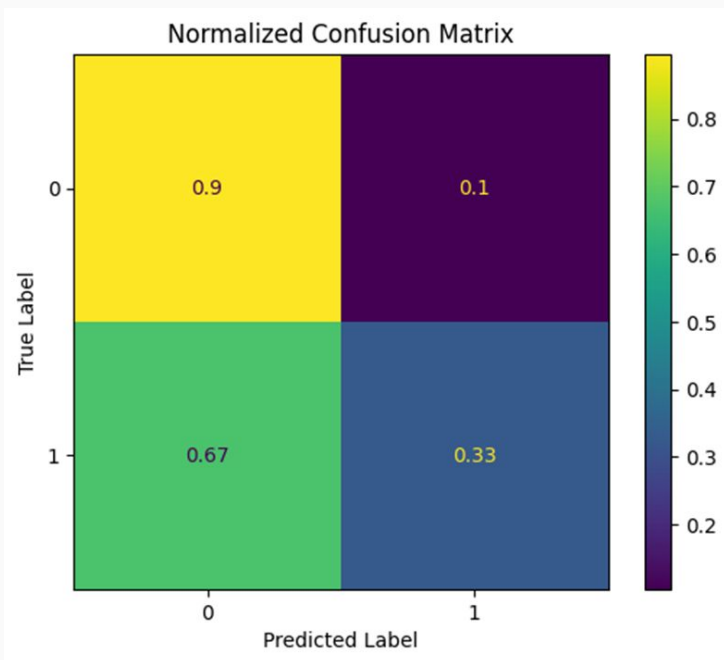
```
model = RandomForestClassifier(n_estimators=50, max_depth=10)
```

```
model.fit(X_train_flat, y_train)
```

```
RandomForestClassifier(max_depth=10, n_estimators=50)
```

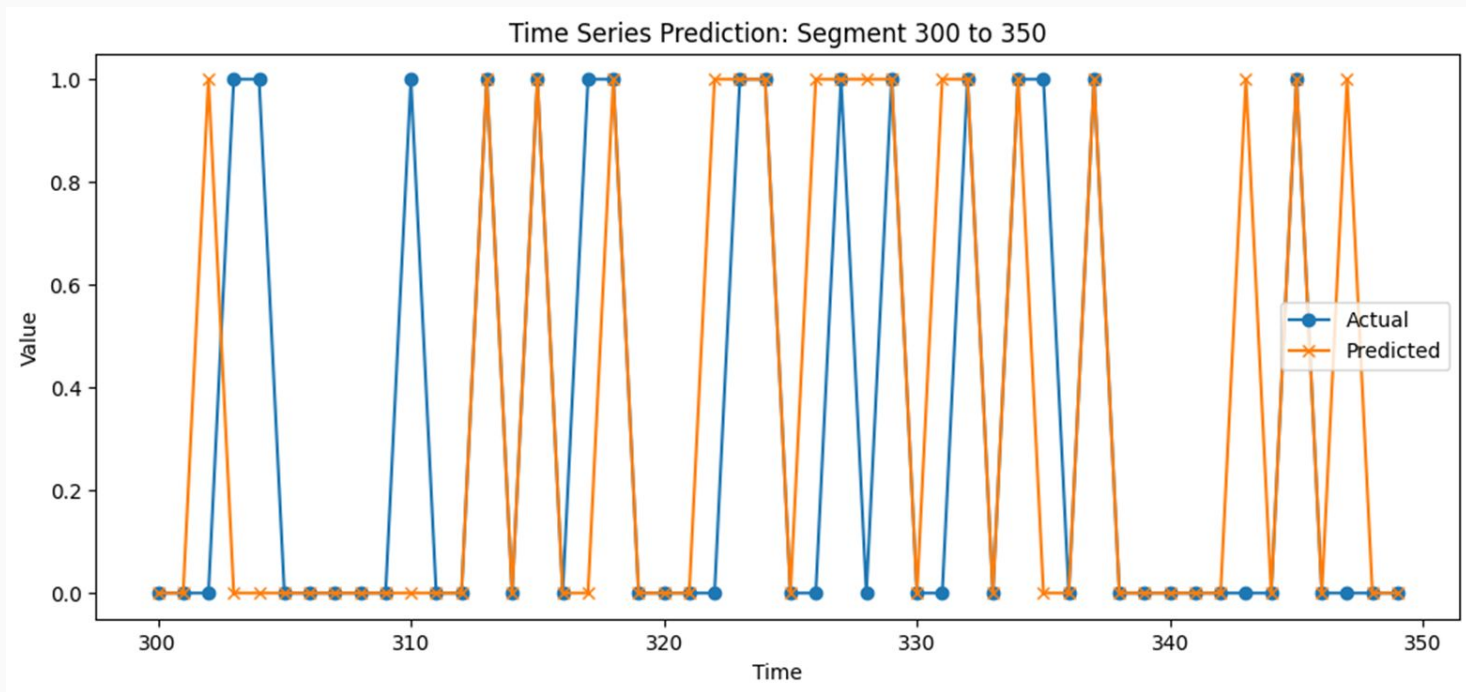


Random Forest

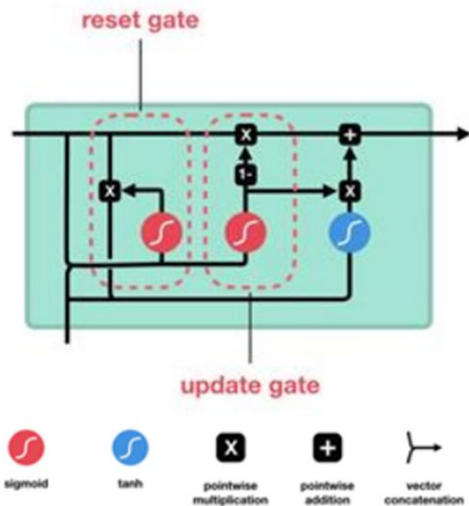


```
Test Accuracy: 0.709375
Precision: 0.625
Recall: 0.7069408740359897
F1 Score: 0.6634499396863691
```

Random Forest



Gated Recurrent Unit (GRU)



```
import torch
import torch.nn as nn

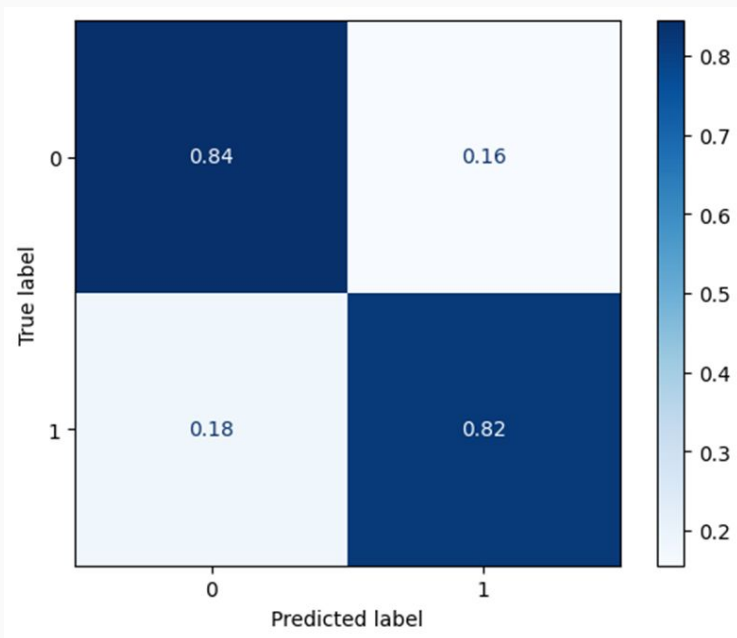
class DeepGRUModel(nn.Module):
    def __init__(self, input_size=6, hidden_size=128, num_layers=3, dropout=0.3):
        super().__init__()

        self.gru = nn.GRU(
            input_size=input_size,
            hidden_size=hidden_size,
            num_layers=num_layers,
            batch_first=True,
            dropout=dropout,
            bidirectional=True # optional but effective
        )

        self.classifier = nn.Sequential(
            nn.Linear(hidden_size * 2, 128), # *2 due to bidirectional
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(64, 1) # output raw logits
        )

    def forward(self, x):
        out, _ = self.gru(x) # out: (B, T, 2*H)
        out = out[:, -1, :] # last time step: (B, 2*H)
        return self.classifier(out) # logits → use BCEWithLogitsLoss
```

Gated Recurrent Unit (GRU)



Test Accuracy: 0.8333333333333334

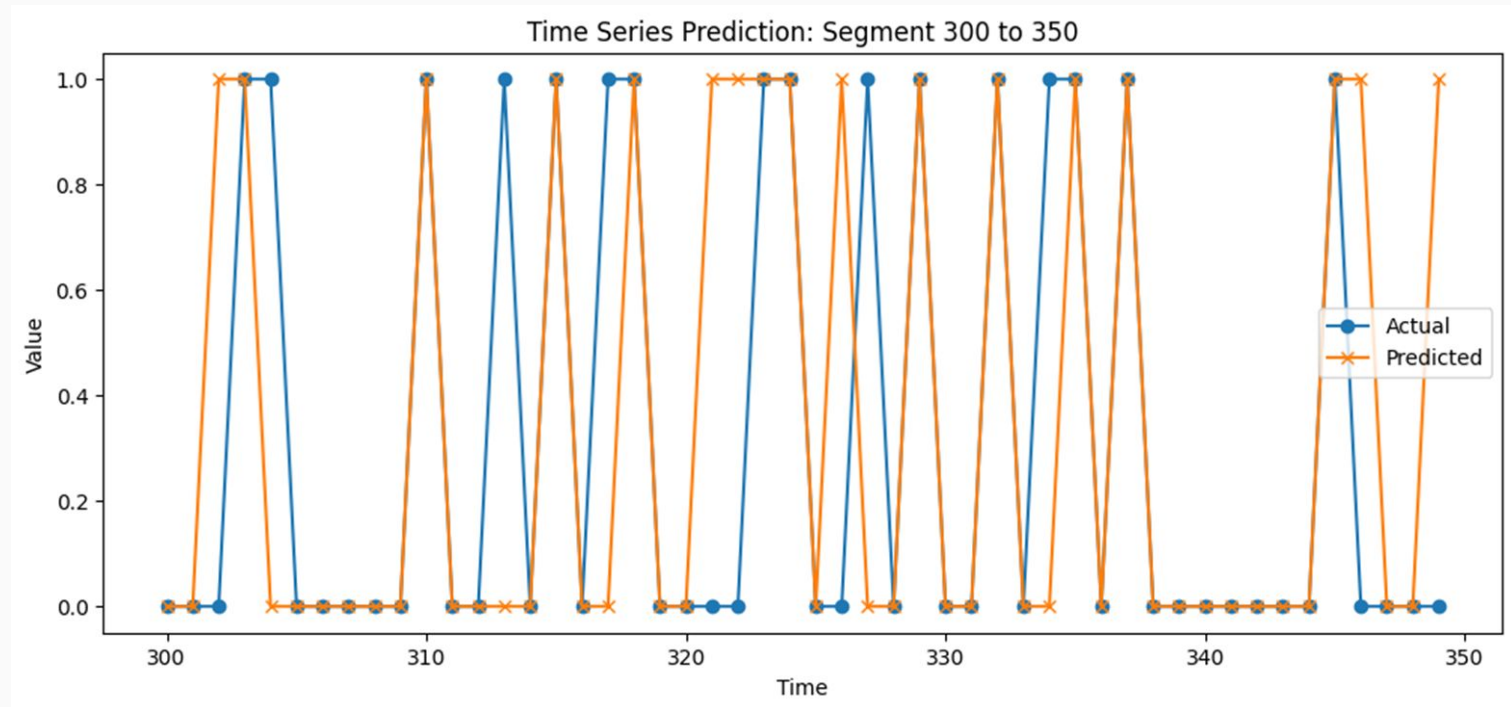
Precision: 0.7813267813267813

Recall: 0.8174807197943444

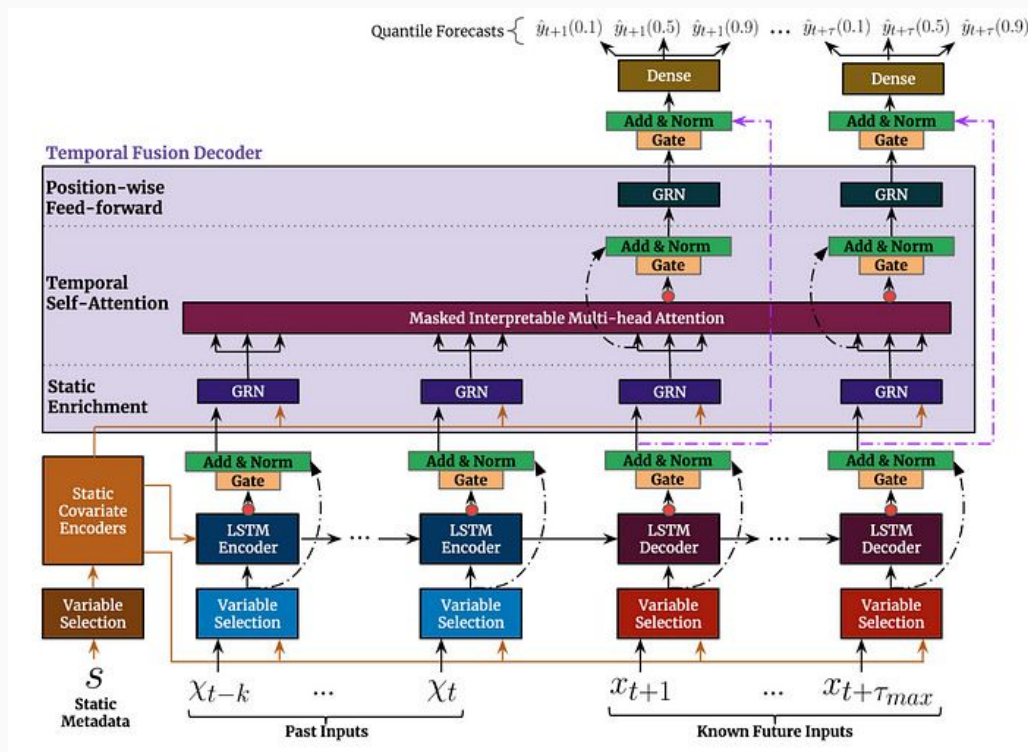
F1 Score: 0.7989949748743719

ROC AUC: 0.9121957149095755

Gated Recurrent Unit (GRU)



Temporal Fusion Transformer (TFT)



Temporal Fusion Transformer (TFT)

```
class GatedResidualNetwork(nn.Module):
    """Light-weight GRN from the TFT paper (without context vectors)."""

    def __init__(self, in_dim: int, hidden_dim: int, out_dim: Optional[int] = None, dropout: float = 0.1):
        super().__init__()
        self.out_dim = out_dim or in_dim
        self.fc1 = nn.Linear(in_dim, hidden_dim)
        self.elu = nn.ELU()
        self.fc2 = nn.Linear(hidden_dim, self.out_dim)
        self.dropout = nn.Dropout(dropout)
        # GLU expects 2xchannels → concatenate with itself
        self.gate = nn.GLU()
        self.skip = nn.Identity() if in_dim == self.out_dim else nn.Linear(in_dim, self.out_dim)
        self.norm = nn.LayerNorm(self.out_dim)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        residual = self.skip(x)
        x = self.fc1(x)
        x = self.elu(x)
        x = self.dropout(x)
        x = self.fc2(x)
        x = self.gate(torch.cat([x, x], dim=-1))
        return self.norm(x + residual)
```

Temporal Fusion Transformer (TFT)

```
class TemporalFusionTransformer(nn.Module):
    """Minimal Temporal Fusion Transformer (sequence-level binary classification)."""

    def __init__(
        self,
        input_size: int = 6,
        hidden_size: int = 64,
        lstm_layers: int = 1,
        attn_heads: int = 4,
        dropout: float = 0.1,
    ) -> None:
        super().__init__()
        self.hidden_size = hidden_size

        # 1. Variable selection / embedding
        self.input_proj = nn.Linear(input_size, hidden_size)
        self.var_gate = GatedResidualNetwork(hidden_size, hidden_size)

        # 2. Sequence encoding (bi-LSTM)
        self.lstm = nn.LSTM(
            input_size,
            hidden_size,
            hidden_size,
            num_layers=lstm_layers,
            batch_first=True,
            bidirectional=True,
        )
        self.bidir_compress = nn.Linear(hidden_size * 2, hidden_size)

        # 3. Temporal fusion block (multi-head self-attention)
        self.attn = nn.MultiheadAttention(hidden_size, num_heads=attn_heads, dropout=dropout, batch_first=True)
        self.attn_grn = GatedResidualNetwork(hidden_size, hidden_size)

        # 4. Sequence-Level output head - returns (B, 1) logits
        self.head = nn.Sequential(
            nn.Dropout(dropout),
            nn.Linear(hidden_size, hidden_size // 2),
            nn.ReLU(),
            nn.Linear(hidden_size // 2, 1), # single Logit *without* squeeze
        )
```

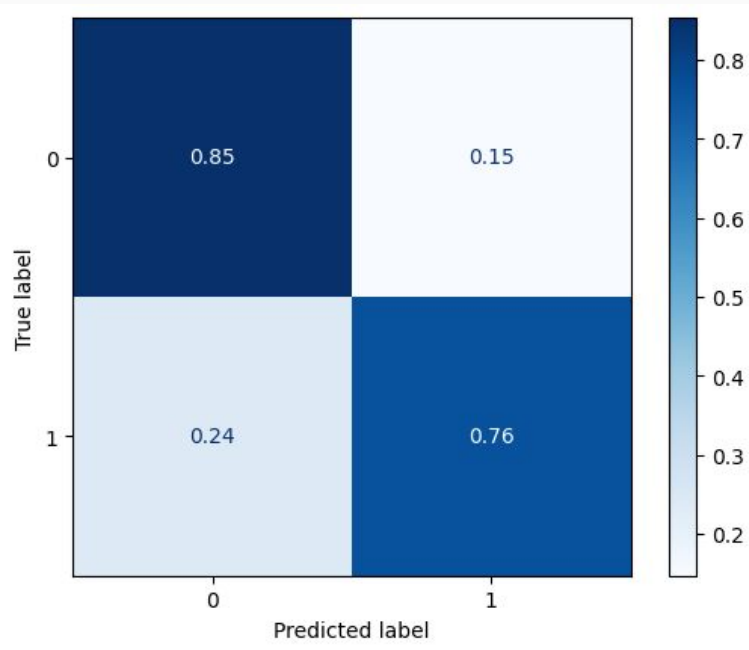
```
def forward(self, x: torch.Tensor) -> torch.Tensor:
    """x: (B, T, F) -> logits: (B, 1)"""
    # Variable selection
    x = self.input_proj(x)
    x = self.var_gate(x)

    # Temporal encoding
    x, _ = self.lstm(x)
    x = self.bidir_compress(x) # (B, T, H)

    # Temporal fusion via interpretable attention
    attn_out, self.last_attn_weights = self.attn(x, x, x)
    x = self.attn_grn(attn_out)

    # Final logit from last step
    logits = self.head(x[:, -1]) # (B, 1)
    return logits # KEEP 2-D so BCEWithLogitsLoss matches y.shape (B, 1)
```

Temporal Fusion Transformer (TFT)



Test Accuracy: 0.8166666666666667

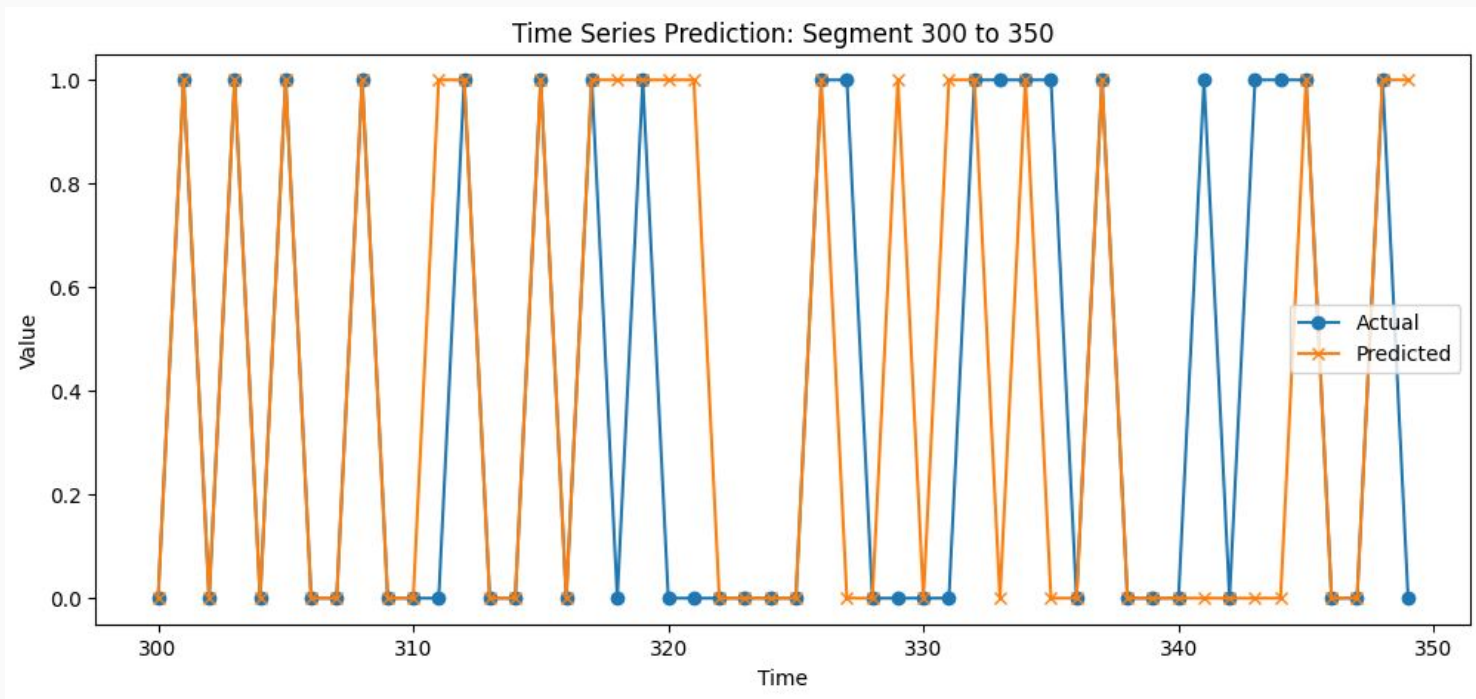
Precision: 0.7795275590551181

Recall: 0.7634961439588689

F1 Score: 0.7714285714285715

ROC AUC: 0.8951237849981316

Temporal Fusion Transformer (TFT)



Conclusion: Results

Models	Accuracy
Flipping a Coin	50%
Time series forest classifier (from sktime)	58%
Conv-LSTM Model	74%
Random Forest	70%
GRU	83%
TFT	81%

Conclusion

- The more complex the model becomes, the better it can capture the temporal dependencies in the accelerometer data.
- GRU outperformed all other models across all the metrics.
- To further increase performance we could ...
 - ... perform more hyperparameter tuning
 - ... use data augmentation for time series

Thank you!

